Hands-on Exam Final Report

Group 1 – Cyberchase

COSC 3341 – Computer Network

11/19/2025

Houston Christian University

Contents

Objective: 3
Configure Switches & Allow for Basic Communication: 3
Addressing and ARP Table Lab 3
Socket Programming UDP Server and Client Program 4
Socket Programming File Transfer Server and Client Program 4
Wireless Router Configuration Lab 5
Hands-On Final 5
Initial Configuration and Reconfiguration 5
Basic OSPF Configuration 6
Summary of OSPF Configuration Steps 6
Task: Configure OSPF Router IDs 6
Task: Verify OSPF Operation 7
Task: Examine OSPF Routes in the Routing Tables 8
Conclusion 8
Task: Pinging everyone (Those who were available) 8
Task: File Transfer with Group 5 9

Objective:

The overall goal of these labs was to set up network switches and routers to enable communication among our group members and between different groups in our class. To achieve this, we completed a series of hands-on networking and programming tasks. On the networking side, we configured switch and router interfaces, set device hostnames, implemented console and privileged access security, and assigned appropriate IP addresses to ensure effective communication within our team and with other groups. We verified connectivity using ping, analyzed ARP behavior to understand how devices discover each other, and troubleshot issues as they arose. In addition to the networking setup, we initially received TCP server and client code, which we modified to UDP to test basic communication. We also completed a wireless router configuration lab where we set up a Wavlink AX3000 router, configured its IP settings and wireless networks, secured the admin account, and switched it to access point mode before verifying wireless connectivity among our group members. Through these steps, we demonstrated that communication could successfully occur from our network to all other networks. Lastly, we wrote a TCP socket program capable of sharing a file across the entire network, demonstrating that a file could be sent and received from one LAN to another, proving end-to-end connectivity and data transfer across the class-wide network.

Configure Switches & Allow for Basic Communication:

In this lab, we configured a Cisco switch to establish basic network communication. The Task included setting the hostname to S1, securing console and privileged EXEC access with a password, enabling password encryption, and adding a warning banner to prevent unauthorized access. We assigned the switch to a management IP address of 192.168.2.100/24 on VLAN 1, brought the interface online, and saved the running configuration. After completing the switch setup, we assigned a laptop with an IP address within the same network to verify connectivity using ping. Finally, we connected both switches with a Cat5 cable and confirmed successful communication between them.

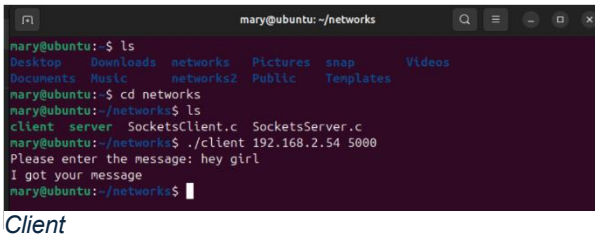
Addressing and ARP Table Lab

In this lab, we learned how IP addressing and MAC-layer communication work together within a local network. We started by assigning each device its specific IP address, subnet mask, and default gateway, then recorded the corresponding MAC addresses to complete the network address table. After setting up the network, we verified connectivity by pinging all computers and switches to ensure every device could communicate successfully. We then explored how ARP functions by clearing ARP tables, pinging devices to regenerate entries, and observing how switches update their MAC address tables. Using arp -a and show mac address-table, we observed how devices learn hardware addresses and how ARP and MAC tables rebuild through network traffic, reinforcing how Layer 2 and Layer 3 interact in real network communication.

Socket Programming UDP Server and Client Program

In this lab, we were provided with working TCP server and client code, and our task was to modify it to use UDP instead. We wrote the socket creation, binding, and communication functions so the server could receive messages from the client side. After updating the code, we tested it by running the UDP server and sending a message from the client to verify successful data transfer. Through this process, we gained a better understanding of how UDP differs from TCP in terms of reliability, speed, and behavior, while also strengthening our skills in network socket programming.

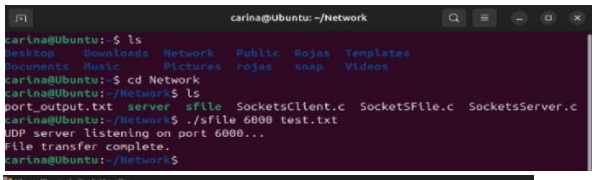
Server

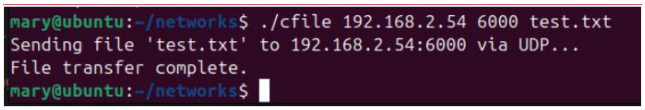


Client

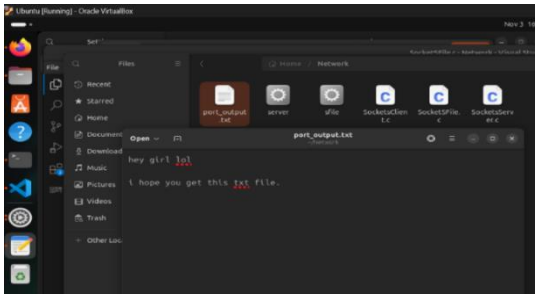
Socket Programming File Transfer Server and Client Program

In this lab, we expanded our UDP server and client code to support file transfer. After converting the original TCP example to use UDP, we further modified the code so the client could read a file, split it into data chunks, and send those chunks to the server using UDP packets. The server was updated to receive each packet and reconstruct the file by writing the incoming data to a new file. Once finished, we tested the program to verify that the entire file could be successfully sent and received across the network. This lab helped us understand how UDP handles large data transfers, manage packet flow manually, and how file transmission works at the application layer.





Client



Server

Wireless Router Configuration Lab

In this lab, we configured a Wavlink AX3000 wireless router and prepared it to function as a wireless access point on our network. We connected a laptop directly to the router, assigned it a static IP, disabled its wireless adapter, and verified connectivity by pinging 192.168.1.1 before accessing the router's login page. After completing the initial setup, we secured the admin account and manually configured the required settings, including assigning the router the IP address 192.168.2.2, creating both 2.4 GHz and 5 GHz SSIDs using our group name, applying the wireless password hcuCOSC3342, disabling the guest network, and switching the device to AP mode. Once the configuration was complete, each group member connected to the new wireless network and verified communication by pinging one another.

Hands-On Final

Initial Configuration and Reconfiguration

As part of the Hands-On Final, we configured a Wavelink AX3000 wireless router to function as a wireless access point on our network. We connected a laptop directly to the console port on the switch and assigned it the IP address 10.10.1.100, following the same subnet as the router's LAN interface, enabling communication and configuration of the device. Accessing the router's CLI through the console port, we set the interface g0/0 to LAN with the IP address 10.10.1.1, while the interface g0/1 was set to WAN and assigned it the IP address 10.100.1.101, with both having 255.255.255.0 as their subnet. These interface settings bring the interfaces online, allowing to router to send and receive traffic on both the WAN and LAN ports. We then configured the wireless settings by assigning a static IP address, 10.10.1.90, combining both 2.4 GHz and 5 GHz SSIDs using our group name, and securing the network with the password hcuCOSC3342. Lastly, we placed the device in Access Point mode, allowing it to integrate into the network seamlessly.

```
Router1(config)#int g0/0
Router1(config-if)#ip address 10.10.1.1.  255.255.255.0
Router1(config-if)#no shutdown
Router1(config-if)#
*Nov 19 15:17:01.187: %LINK-3-UPDOWN: Interface GigabitEthernet0/0,
changed state to down
Router1(config-if)#description Connection to LAN
Router1(config-if)#end
Router1#c
*Nov 19 15:17:24.667: %SYS-5-CONFIG_I: Configured from console by
consoleon
Router1#config t
Enter configuration commands, one per line.  End with CNTL/Z.
Router1(config)#int g/1  0/1
Router1(config-if)#ip address 10.100.1.101 255.255.0.0
Router1(config-if)#no shutdown
```

Configuration LAN & WAN Interfaces 1

Basic OSPF Configuration

Summary of OSPF Configuration Steps

The OSPF configuration process includes preparing the router, assigning interface addresses, and enabling dynamic routing to exchange routes with neighbors. First, configure the router with its basic settings and interface IP addresses for both the LAN (10.10.1.1/24) and WAN (10.100.1.101/16) networks. DNS lookups are disabled to simplify command-line operation, and interfaces are checked to confirm they are active and properly configured. After connecting the router to network switches, enable OSPF routing with Process ID 1. The router then advertises its LAN and WAN networks via OSPF network statements in Area 0, allowing it to share and receive route information across the network dynamically. Finally, save the configuration to ensure it remains after reboots, completing the OSPF setup.

```
Routing Protocol is "ospf 1"
  Outgoing update filter list for all interfaces is not set
  Incoming update filter list for all interfaces is not set
  Router ID 10.100.1.101
  Number of areas in this router is 1. 1 normal 0 stub 0 nssa
  Maximum path: 4
  Routing for Networks:
    10.10.1.0 0.0.0.255 area 0
    10.100.1.0 0.0.0.255 area 0
```

OSPF Configuration

Task: Configure OSPF Router IDs

In this step, we configured OSPF on the router and prepared it to join an OSPF network. The first step was to create a stable OSPF Router ID by configuring a Loopback interface. We entered interface configuration mode and created Loopback0, assigning it the unique IP address 10.1.1.1/32, making sure it did not conflict with any existing interface addresses. This Loopback address later became the router's permanent OSPF Router ID. Following the instructions and the handwritten note, we saved the configuration with the copy running-config startup-config command before reloading the device to prevent losing any changes.

Next, we restarted the OSPF process so the router would adopt the new Router ID. Since OSPF does not automatically update the Router ID after adding a Loopback interface, the router had to be restarted for it to start using 10.1.1.1. After the restart, we verified OSPF operation with commands such as show ip protocols, show ip ospf, show ip ospf interface, and show ip ospf neighbor. The verification confirmed that the router was correctly using the new Router ID and that the networks 10.10.1.0/24 and 10.100.1.0/16 were assigned to Area 0. No OSPF neighbors appeared at this point, as expected, since the other routers in the topology had not yet been configured or powered on.

We then examined the router's routing table using the show ip route command to check for any OSPF-learned routes. As expected, no OSPF routes were present because no adjacency had yet been formed, and only connected and local routes were visible. This confirmed that the

router's OSPF configuration was correct, but that it needed neighboring routers to be online to start learning OSPF routes. Finally, we restarted the OSPF process with the clear ip ospf process command, confirming the reset and ensuring the router fully adopted the Loopback-based Router ID. After the reset, the router operated correctly with Router ID 10.1.1.1, and all OSPF processes were running normally and ready for future neighbor formation.

```
Router1(config)#int loopback0
Router1(config-if)#ip address 10.1.1.1 255.255.255.255
Router1(config-if)#end
```

OSPF Loopback

Task: Verify OSPF Operation

In this step, we verified that OSPF was operating correctly on Router R1 (our router) and confirmed that OSPF-learned routes were populating the routing table as expected. After completing the router's WAN and Lan interface configuration, we enabled OSPF process 1 and advertised the LAN and WAN networks into Area 0 using the network statements visible in both the configuration and the logs. We began by running the "show ip ospf neighbor" command to verify that R1 and successfully formed an adjacency with Group 5's router (R5). The output confirmed that R1 reached the OSPF neighbor using next-hop address 10.100.5.101 on terface GigabitEthernet0/1, and the neighbor was tine the FULL/DR state, indicating a fully established adjacency:

```
Neighbor ID     Pri   State           Dead Time   Address         Interface
10.5.5.5          1   FULL/DR         00:00:32    10.100.5.101    GigabitEthernet0/1
Router1#show ip ospf neighbors□□□□□□□□□□□□□□□□□□□□□□□□ping 10.10.5.101      □□□□□□
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.5.101, timeout is 2 seconds:
.....
Success rate is 0 percent (0/5)
Router1#ping 10.10.5.101□□□□□□□□100.5.101□□□□□□□
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.100.5.101, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/4 ms
Router1#ping 10.10.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.1.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/1 ms
```

putty-final-router-ping-test-1 1

Using the "show ip protocols" command, we confirmed that R1 was running OSPF process 1, using Router ID 10.10.1.1, and advertising both the LAN and WAN networks in Area 0. The logs show R1 was correctly routing for:

- 10.10.1.0/24 – LAN network
- 10.100.1.0/016 – WAN network

Task: Examine OSPF Routes in the Routing Tables

Next, we examined R1's routing table using "show ip route" to confirm that OSPF-learned networks were present. In the routing table, OSPF routes are marked with the code "O", and R1 correctly installed the routes learned from R5.

```
      10.0.0.0/8 is variably subnetted, 6 subnets, 3 masks
C        10.1.1.1/32 is directly connected, Loopback0
C        10.10.1.0/24 is directly connected, GigabitEthernet0/0
L        10.10.1.1/32 is directly connected, GigabitEthernet0/0
O        10.10.5.0/24 [110/2] via 10.100.5.101, 00:16:49, GigabitEthernet0/1
C        10.100.0.0/16 is directly connected, GigabitEthernet0/1
L        10.100.1.101/32 is directly connected, GigabitEthernet0/1
```

OSPF learned routes verification

This confirmed that routes from Group 5's router were not only being received, but were also being chosen as valid path based on OSPF cost (110/2). The table also showed directly connected networks.

Conclusion

Task: Pinging everyone (Those who were available)

Our client on our network was able to successfully communicate with and ping to a client in the following groups: group 3 and 5

```
PS C:\Users\Matthew> ping 10.10.5.125

Pinging 10.10.5.125 with 32 bytes of data:
Reply from 10.10.5.125: bytes=32 time=123ms TTL=62
Reply from 10.10.5.125: bytes=32 time=144ms TTL=62
Reply from 10.10.5.125: bytes=32 time=200ms TTL=62
Reply from 10.10.5.125: bytes=32 time=109ms TTL=62

Ping statistics for 10.10.5.125:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 109ms, Maximum = 200ms, Average = 144ms
PS C:\Users\Matthew>
```

Client from group 1 pinging a client from group 5

```
PS C:\Users\Matthew> ping 10.10.3.11

Pinging 10.10.3.11 with 32 bytes of data:
Reply from 10.10.3.11: bytes=32 time=18ms TTL=62
Reply from 10.10.3.11: bytes=32 time=9ms TTL=62
Reply from 10.10.3.11: bytes=32 time=11ms TTL=62
Reply from 10.10.3.11: bytes=32 time=9ms TTL=62

Ping statistics for 10.10.3.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 9ms, Maximum = 18ms, Average = 11ms
PS C:\Users\Matthew>
```

Client from group 1 pinging a client from group 3

```
PS C:\Users\Matthew> ping 10.10.5.123

Pinging 10.10.5.123 with 32 bytes of data:
Reply from 10.10.5.123: bytes=32 time=63ms TTL=62
Reply from 10.10.5.123: bytes=32 time=66ms TTL=62
Reply from 10.10.5.123: bytes=32 time=186ms TTL=62
Reply from 10.10.5.123: bytes=32 time=96ms TTL=62

Ping statistics for 10.10.5.123:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 63ms, Maximum = 186ms, Average = 102ms
PS C:\Users\Matthew>
```

Client from group 1 pinging a different client from group 5

Task: File Transfer with Group 5

Our client successfully transferred a file from our network to a client a group's network. We were able to operate as both the server and client during testing, allowing us to send and receive a text file to and from Group's network without any issues.

```
mary@ubuntu:~/final$ ./client 10.10.5.125 5000 test.txt
Preparing to send file: test.txt (46 bytes)
Connected to server.
Sending file header...
Header acknowledged. Starting data transfer...
Progress: 46 / 46 bytes (100.00%)
File transfer complete.
```

Client: Mary acting as the client and sending "test.txt" to James (group 5)

```
mary@ubuntu:~/final$ ./server 5000
TCP File Server running on port 5000... Waiting for connection.
Client connected. Starting file transfer.
Receiving file: other.txt (24 bytes)
Progress: 24 / 24 bytes (100.00%)
File transfer complete. Saved as: _received_tcp_other.txt
mary@ubuntu:~/final$
```

Server: Mary acting as the server and receiving a txt file from James (group 5)